

Mnemosyne Performance & Refactoring Blueprint

A no-compromise plan to massively improve the performance and architecture of the Mnemosyne memory compiler — while preserving strict byte-parity, local-first defaults, and all seven §31 invariant rails.

Field	Value
Document	Mnemosyne Performance & Refactoring Blueprint
Version	1.1 (expanded — supersedes 1.0 draft of the same date)
Status	Proposed — not yet ratified; no production claims
Author	Engineering planning (agent-drafted)
Date	2026-07-08
Scope	End-to-end performance & structural refactoring of the Mnemosyne v2 runtime, including the production stack runtime, the journal/projection substrate, and memory-capability upgrades drawn from the 2025–2026 open-source state of the art
Governing constraint	Strict byte-parity everywhere — the deterministic retrieval/calibration kernels remain the bit-identical correctness oracle. No optimization may change oracle numerics.
Deployment targets	Both weighted equally: local-first CPU (in-memory / SQLite) and hosted Postgres + provider/GPU
Companion docs	docs/blueprint/Mnemosyne-v2-Build-Blueprint.md , docs/ROADMAP-T0-100.md , docs/superpowers/specs/2026-07-05-performance-program-design.md , docs/superpowers/specs/2026-07-01-native-acceleration-design.md , docs/decisions/SECTION-17-OPEN-QUESTIONS.md , infra/PERF-RUNTIME.md , .planning/codebase/CONCERNS.md , eval/latency/README.md
Non-goals	Relaxing §31 rails; making an optional provider a silent default; replacing Postgres as the production scale backend; changing the evidence-ledger source-of-truth model; reopening settled SECTION-17 decisions

Evidence discipline. Every measured figure in this document is an existing local/eval artifact or an external published benchmark, cited as such. Nothing here is a production sign-off. All projected speedups are labelled *projected* and must be proven by the benchmark plan in §11 before any status doc is updated, per .planning/ACTIVE-GOAL-OPERATING-CONTRACT.md .

v1.1 changelog. Adds: the production-runtime baseline and the now-unblocked deferred P1/P2 flips (§3.5, §7.9 — the single largest measured win in the whole program); reconciliation with the standing P0–P5 performance program and native-acceleration phases so this blueprint extends rather than duplicates them (§4.6); corrections to two stale bottleneck facts (connection pooling landed in P3; embedding caches exist on the hashing and SQLite paths — §5 B6/B8); the embedding_partition='none' HNSW index gap and Postgres server tuning (§7.4); the journal & projection performance package (§7.10); the structural architecture refactor register beyond cli.py (§7.11); memory-capability upgrades from the SOTA survey with license governance (§9.1–§9.2); Wave 0 and Wave E in the roadmap (§10); and the explicit measurement-gap register (§11.8).

1. Executive summary

Mnemosyne is already fast on its proven warm path: the Wave-3 long-lived-server bench measured a **fast-path P95 of 149.5 ms** against a 300–400 ms SLO, with recall@k 0.977, nDCG@k 0.983, and calibration ECE 0.0063. The system is not slow. The goal of this blueprint is to make it *decisively* fast — to turn the current comfortable SLO margin into a multiplicative lead — without giving up the two properties that make Mnemosyne unusual and trustworthy: **bit-exact reproducibility** of its retrieval and calibration math, and a **zero-dependency local-first default**.

The central engineering thesis is a clean separation of concerns that lets us have both speed and exactness at once:

Speed comes from the candidate-generation and systems layers — indexing, caching, batching, provider colocation, native-default execution, GIL release, and connection pooling. Exactness is preserved by keeping the final scoring, fusion, and calibration on the bit-identical f64 kernels, which remain the untouched correctness oracle.

This framing is what makes “massively faster *and* no-compromise” achievable rather than contradictory. Approximate nearest-neighbour indexes (HNSW, DiskANN) already sit *outside* the byte-parity oracle — they generate candidates; they do not define truth. We can therefore be aggressive about candidate generation (quantized indexes, iterative scans, disk-resident graphs) while the exact cosine /RRF/MMR kernels re-score the shortlist and drive calibration unchanged. Every recommendation in this document is tagged **PARITY-SAFE** (no oracle numerics change), **APPROX-LAYER** (changes only candidate generation, which is already approximate and separately validated), or **OPT-IN / RECAL** (changes numerics and is therefore gated behind explicit configuration plus recalibration, never a silent default).

The ten highest-leverage moves, in priority order:

1. **Execute the committed-but-dormant production runtime flip** (PARITY-SAFE; pure ops). Host-Metal LLM serving and the colima VM resize are already committed as the default topology (`infra/PERF-RUNTIME.md`) but have never been applied to the live stack. Measured motivation: in-VM `qwen3:4b` ≈ 0.5 tok/s vs host-Metal ≈ 24 tok/s — a **$\sim 48\text{--}55\times$ measured gap** on every consolidation role call, which today costs 95–320 s per call. The stated precondition (pending Tier-B production evidence capture) **completed on 2026-07-07** (`planning/STRICT-BLUEPRINT-PARITY-AUDIT.md` , capture-bc10), so the flip is unblocked pending operator confirmation. One guarded script run is the largest single performance win available anywhere in this document (§3.5, §7.9).
2. **Make the native Rust kernels the standard tier and gate their wheels in CI** (PARITY-SAFE). The scalar byte-parity kernels are already measured at $41\times$ (lexical), $12\times$ (PPR), $\geq 10\times$ (dense) over the pure-Python fallback while bit-identical — but wheels are non-gating today, so most installs silently run the slow path. Realizing acceleration that already exists is the cheapest engine-side win.
3. **Release the GIL around native compute and make parallel channels the default on capable tiers** (PARITY-SAFE). `Python::detach` + rayon-across-items already-safe kernels, plus overlapping the dense/lexical/graph channels, converts serial CPU into parallel CPU with no numeric change.
4. **Backfill null embeddings, close the `embedding_partition='none'` index gap, and cap the Postgres dense fallback** (APPROX-LAYER for the index; PARITY-SAFE for scoring). The current `embedding IS NULL` path embeds many rows in Python *per query*; separately, rows whose `embedding_partition` is `'none'` — the schema default (`sql/schema.sql:42`) — are covered by **no HNSW index at all** (the indexes are partial over `public / private` only, `sql/schema.sql:50–51,89–90`) and silently fall back to exact scans.
5. **Complete the caching lattice: content-hash real-model embedding cache, result cache, and calibration cache** (PARITY-SAFE — caching returns identical bytes). The hashing-embedding path is already LRU-cached (`text.py:96`) and SQLiteEngine carries the subject-scoped A1 `embedding_cache` table; the missing piece is a (`model_id` , `model_rev` , `text_sha256`)-keyed cache in front of `HttpEmbeddingProvider` for the Local/Postgres paths, where embedding inference dominates warm `fast_path_total` .
6. **Adopt a production-grade embedding server (HuggingFace Text-Embeddings-Inference) with token-based dynamic batching, colocated with the engine** (APPROX-LAYER only if the model changes; PARITY-SAFE if the model/precision is held constant) — and run the already-built `rust/mneme-providers` sidecar bake-off that the native-acceleration program requires before any default change.
7. **Fast approximate shortlist + exact rescoring on Postgres**: `halfvec` HNSW index and pgvector 0.8 iterative scans for candidate generation, then exact `f64` cosine rescoring on the shortlist (APPROX-LAYER for the index, PARITY-SAFE for the returned scores). Optional `pgvectorscale` StreamingDiskANN for large tenants. Plus long-missing Postgres server tuning (`work_mem` , `effective_cache_size` , `hns.ef_search`) — today only `shared_buffers / maintenance_work_mem` are set (`infra/docker-compose.prod.yml:109–110`).
8. **Parallelize and batch the consolidation pipeline** (PARITY-SAFE). The five consolidation roles run strictly sequentially per job (`consolidation.py:462–519`) on top of the slow LLM path; role parallelism multiplies with move #1's per-call win.
9. **Ship the journal & projection performance package** (PARITY-SAFE): snapshotting, blue/green validated projection rebuilds with a Rust fold + COPY writer, micro-batched appends, gap-safe sequencing, and per-tenant partitioning that *strengthens* crypto-shredding while it speeds rebuilds (§7.10).
10. **Decompose the 19,529-line `cli.py` monolith and execute the structural refactor register** (PARITY-SAFE; structural): provider-contract unification (the `/embed + /rerank` contract is implemented three times in two languages), schema single-source-of-truth migration discipline, and the pipeline-Protocol guardrail (§7.7, §7.11).

Projected outcome (to be proven, not assumed): the runtime flip alone converts consolidation role calls from $\sim 100\text{--}320$ s to a projected 2–5 s; native-default + GIL-release + caching should compress warm-path P95 well below 100 ms on the local path; colocated dynamic-batched embedding + pooled/quantized Postgres candidate generation should hold P95 under load while raising throughput several-fold; and the CLI decomposition should cut cold-start import time and review cost materially. The phased roadmap in §10 sequences these behind explicit exit gates so no wave lands without a benchmark and a parity proof.

2. How to read this document

The blueprint is organized so an implementer can pick up any single item and execute it in isolation:

- **§3–§6** establish the baseline, the constraints, the ranked bottleneck inventory, and the performance-architecture thesis. Read these once. §3.5 and §4.6 (new in v1.1) situate the blueprint against the live production stack and the standing performance programs.
- **§7** is the core: eleven layer-by-layer refactoring/acceleration plans. Each names the exact files, the change, the parity tag, the projected gain, the risk, and the verification. These are the work items.
- **§8** is the language/method strategy (Rust vs Python vs SQL vs push-down, free-threaded CPython, and the precise rule for when — never, for the oracle — parity may be relaxed).
- **§9** maps specific external projects and papers to specific Mnemosyne changes with a parity verdict for each; §9.1 adds license governance; §9.2 adds the memory-capability upgrades (quality $\times$ speed) drawn from the agent-memory state of the art.
- **§10–§13** are the phased roadmap, benchmark plan, risk register, and file-level change appendix.

Every work item carries one of three parity tags, defined once here:

Tag	Meaning	Gate
PARITY-SAFE	Returns byte-identical results to the current oracle. No numeric change anywhere.	Existing parity tests must stay green; no recalibration needed.
APPROX-LAYER	Changes only <i>candidate generation</i> (which ANN already makes approximate). Final scoring/calibration still runs the exact kernel on the shortlist.	Recall@k / nDCG@k SLO gate on the eval harness; oracle untouched.
OPT-IN / RECAL	Changes numerics that reach the oracle (e.g. truncated or quantized <i>stored</i> embeddings, model swaps, new ranking legs).	Off by default; explicit config; full recalibration + ECE re-proof before any default flip.

3. Current performance baseline (measured evidence)

These are the existing, in-repo measurements. They are the starting line, and several carry honest caveats that this blueprint intends to remove.

3.1 Headline SLOs (Wave-5 definitive run, per `README.md` / `docs/ROADMAP-T0-100.md`)

SLO	Measured	Target	Result
recall@k	0.977	≥ 0.80	PASS
nDCG@k	0.983	≥ 0.80	PASS
G2 token-efficiency lift	+0.208 @ 7% tokens	≥ +0.15	PASS
Poison-block rate	1.0	≥ 0.95	PASS
Calibration error (ECE)	0.0063	≤ 0.05	PASS
Fast-path P95 (warm + serial)	149.5 ms	≤ 300–400 ms	PASS

Interpretation. Quality and calibration have large headroom; they are not the constraint. Latency has a ~2× margin against SLO on the warm path. The opportunity is not “pass the SLO” (already done) but “collapse the latency and raise throughput so the SLO is met with an order-of-magnitude margin under concurrent load.”

3.2 Warm-path decomposition (Wave-3 bench, `eval/latency/`)

The Wave-3 bench corrected a real Wave-2 measurement artifact: Wave-2 spawned a fresh `python -m mnemosyne.cli ... search` subprocess *per call*, charging ~200–300 ms of interpreter/import cold-start and a cold model round-trip to every request. Against a warm long-lived server the honest decomposition is:

- `embed_call_only` — the isolated warm HTTP round-trip to the real embedding model (`BAAI/bge-small-en-v1.5` , 1024-dim). **This dominates `fast_path_total` .**
- `engine_only` — one warm in-process hybrid retrieve (dense + lexical + graph, RRF fuse, MMR, calibrate, budget). Small relative to embedding.
- `fast_path_total = embed + engine` .

Interpretation. On the warm path, *the model inference and its transport are the latency*, not the engine. This is why the embedding-provider layer (§7.2) and caching (§7.8) are ranked so highly. The engine’s own kernels are already cheap in Python and become negligible once native (§7.1).

3.3 Known bottlenecks already catalogued (`.planning/codebase/CONCERNS.md`)

The repo’s own concerns audit independently identifies four performance bottlenecks, which this blueprint adopts as confirmed starting facts:

- Local retrieval scans candidate sets in-process in Python** unless the native extension is present.
- Postgres dense fallback can embed many rows per query** on the `embedding IS NULL` path — an unbounded per-query CPU path.
- Embedding service dominates warm fast-path latency on CPU** — each query pays model inference plus HTTP/JSON overhead unless cached, batched, or colocated.
- Local runtime-state file writes are single-writer** — concurrent shared-engine searches contend on atomic-replace metric persistence.

3.4 External anchors (for context, not targets)

Published 2026 agent-memory benchmarks place Zep/Graphiti retrieval at a **P95 ≈ 300 ms with no LLM calls in the retrieval path**, and Mem0 at a median ≈ 0.708 s / P95 ≈ 1.44 s. Mnemosyne’s 149.5 ms warm fast-path P95 is already competitive-to-leading among comparable systems. The bar this blueprint sets is to make that lead structural and durable under concurrency, not merely to match peers.

3.5 Production runtime baseline — the committed-but-dormant flips (new in v1.1)

The engine numbers above are measured on the *host*. The live production stack runs inside a **4 vCPU / 10 GiB colima VM hosting ~19 containers**, and its two dominant cost centers dwarf every engine-level number in this document:

Cost center	Measured today	Committed fix (not yet applied)	Projected after flip
In-VM CPU role-LLM (qwen3:4b via ollama)	≈ 0.5 tok/s; 95–133 s per trivial role call; consolidation role timeouts at 280–320 s	Host-Metal serving via <code>host-llm-proxy</code> relay is the committed default topology (<code>infra/docker-compose.prod.yml:405–415</code> ; in-VM ollama demoted to the opt-in <code>in-vm-llm</code> profile)	≈ 24 tok/s measured on host Metal — ~48–55x measured ; role calls projected 2–5 s
VM topology / resource contention	4 vCPU / 10 GiB for ~19 containers; declared mem ceilings (ollama 6g + embedder 4g + keycloak 1g + ...) exceed the VM	Colima resize to 6 vCPU / 12 GiB + committed per-service <code>cpus / mem_limit</code> ceilings (postgres 2c/2g, embedder 2c/4g, role-http 2c/1g)	Removes the multiplier on <i>everything</i> ; unmeasured until flipped

Both fixes are **already committed** with guarded apply/rollback scripts (`infra/scripts/apply-perf-runtime.sh` , refuses to act without `MNEMO_CONFIRM=1`). They were deliberately deferred because a stack restart invalidates in-flight Tier-B evidence capture (`infra/PERF-RUNTIME.md` §”When to run”). **That precondition is now satisfied**: the Tier-B capture (capture-bc10) completed 2026-07-07 with `release-audit ok:true` and `production-evidence-verify ok:true` . §7.9 turns this into the program’s Wave 0.

Two consequences for reading everything else in this blueprint: (a) all committed e2e numbers were captured on the *un-resized* runtime and under documented ambient host load (the 2026-07-05 delta run itself flags ~10 load-average contamination, with p95 tails inflated up to +252% on untouched paths) — they are directionally sound but not clean regression baselines; (b) engine-side wins land on a runtime whose dominant costs are elsewhere until Wave 0 executes. **Order of operations matters: flip the runtime first, then re-baseline, then attribute engine wins honestly.**

4. Governing constraints & invariants (the “no-compromise” boundary)

Performance work that violates any of the following is out of scope by construction. These are the guardrails that keep “faster” from becoming “different.”

4.1 Strict byte-parity of the deterministic oracle

The native kernels carry an explicit, unusually strict parity contract (`rust/mnemosyne-native/src/lib.rs` , `dense.rs`): every kernel is **bit-identical** to its pure-Python counterpart. The dense kernel does not merely “sum products” — it reproduces **CPython 3.12’s Neumaier (compensated Kahan–Babuška) summation** exactly, because since gh-100425 builtin `sum()` over floats is compensated, not naive. The allowed float ops are `+ - * / sqrt abs` ; **no fused multiply-add, no SIMD horizontal reductions, no transcendentals**, because each would perturb the last ULP and break parity with the Python reference.

Consequence for this blueprint. SIMD distance libraries (SimSIMD, BLAS-backed cosine) and FMA cannot be used *inside the oracle kernel* — they would change results. This is not a limitation to fight; it is the reason the parity proof holds. The blueprint therefore routes all SIMD-class acceleration into the **candidate-generation (APPROX) layer**, where approximation is already expected and separately validated, and keeps the exact scalar kernel as the shortlist re-scorer and calibration reference. Where a SIMD kernel could still help *without touching the oracle* (e.g. an approximate pre-filter that only reorders which candidates get exactly re-scored), it is allowed and tagged APPROX-LAYER.

4.2 Local-first, zero-dependency default

The package must continue to start and run with a single core dependency (`cryptography`), no network, no Postgres, and no model server. Every provider-based acceleration (TEI, ONNX, GPU) is an **explicit opt-in**, never a silent runtime fallback (`.planning/codebase/ARCHITECTURE.md` anti-pattern: “Provider as Implicit Dependency”).

4.3 The seven §31 invariant rails

Bounded supersession (R1), corroborated deletion (R2), bounded pruning (R3), monotonic trust (R4), external-only reward (R5), retrieved-text-is-data (R6), bounded cadence (R7). No performance change may relax a rail. In particular, R6 (sanitize every retrieved span) sits on the read hot path and must remain — caching must cache *post-sanitization* results so the rail is never bypassed for speed.

4.4 Backend parity and substitutability

Local, SQLite, and Postgres engines must remain contract- and parity-equivalent (`tests/test_shared_engine_contract.py`). Cross-backend behavior goes in shared modules (`pipeline.py` , `algorithms.py` , `models.py`); backends expose only storage primitives. A speedup added to one backend that cannot be matched in the shared contract is a regression in disguise.

4.5 Evidence and false-completion controls

No optimization may weaken the production-evidence custody path or introduce a “fast path” that skips a security/provenance/redaction boundary. Redaction and access-policy helpers (`access_policy.py` , `evidence_redaction.py`) are trust boundaries; any new fast read path must route through them. Honeytoken canaries (`honeytokens.py`) must be seeded across **every new boundary this blueprint introduces** — every new cache tier, the profile projection, and any native or sidecar surface — so a cross-tenant leak through a performance feature fires the same alarm as one through the engine.

4.6 Relationship to the standing programs (new in v1.1)

This blueprint **extends two live, partially-shipped programs; it does not restart them**. Implementers must read both specs before picking up any §7 item, and must not re-do landed work:

- **Performance program P0–P6** (`docs/superpowers/specs/2026-07-05-performance-program-design.md`). Already landed on `main`: P0 measure-first baselines (`tests/benchmarks/baselines.json` , `eval/latency/reports/perf-delta-20260705.md` — MCP cold start 112.6→88.3 ms self-time, −21.6%); P1 runtime right-sizing (committed, unapplied — §3.5); P2 host-Metal LLM topology (committed, unapplied); P3 engine hot-path surgery (connection pool `postgres_engine.py:121,183` ; candidate-scan memoization; RRF shallow-reconstruction; stored `lexeme TSVECTOR + GIN`, `sql/schema.sql:43,52` ; batch embedding client; MCP lazy imports); P4 native wave 2 (`ppr_power_iteration` , measured 12.4x); P5 capability tiering (`capability.py`). Git lane letters map to spec phases: B1–B3 = P3, C = P4, D = P5, E = P1.
- **Native-acceleration program** (`docs/superpowers/specs/2026-07-01-native-acceleration-design.md` , spec commit `31c50a5`). Phases 0–3 are done (seam hardening; kernels measured lexical 41x, prepacked dense 82.5x, hashing ~4.9x; SQLiteEngine with packed-BLOB dense ~43x; sidecar reranker built). Phases 4–6 (Rust front-end, ANN/quantized tiers, IVM) are **explicitly gated on measurements** — this blueprint adopts those gates verbatim rather than re-deciding.
- **Settled decisions are binding** (`docs/decisions/SECTION-17-OPEN-QUESTIONS.md` , 7/7 resolved). In particular: OQ3 chose **salsa-style memoization on the existing queue** and explicitly rejected `pg_ivm`/differential-dataflow for now — §7.10 builds on that choice; OQ1 chose cached-PPR on the fast path with live PPR async-deep only; OQ7 chose asymmetric capability mediation to avoid a read tax. This blueprint routes any *new* open question through the same decision-memo/ADR pattern (Appendix C) rather than reopening settled ones.
- **The uniform wiring contract holds**: every change here ships **additive, default-off / shadow-first, byte-identical when inactive, no new core deps** (`docs/ROADMAP-T0-100.md` Tier-A discipline), with defaults flipped only behind the §11 gates.

5. Bottleneck inventory (ranked, with evidence)

Each row links a concrete, evidenced bottleneck to the section that fixes it and the parity tag of that fix. Rows B15–B20 are new in v1.1; B6 and B8 are corrected against current `main` (their v1.0 statements were stale).

#	Bottleneck	Evidence	Primary fix	Parity tag
B15	In-VM CPU role-LLM: ~0.5 tok/s; 95–320 s per consolidation role call — the largest cost center in the system by 2–3 orders of magnitude	<code>perf-program spec §1 rank 1</code> ; <code>infra/PERF-RUNTIME.md</code> ; <code>docker-compose.prod.yml:405–415</code>	§7.9 execute committed host-Metal flip (Wave 0)	PARITY-SAFE (ops)
B16	VM topology: 4 vCPU/10 GiB for ~19 containers — declared container ceilings exceed the VM; contention multiplies everything	<code>perf-program spec §1 rank 2</code> ; <code>infra/PERF-RUNTIME.md</code>	§7.9 execute committed VM resize + ceilings (Wave 0)	PARITY-SAFE (ops)
B1	Embedding inference + HTTP/JSON dominates warm <code>fast_path_total</code>	<code>eval/latency</code> decomposition; CONCERNS §Perf	§7.2 provider layer, §7.8 caching	PARITY-SAFE (cache/colocate); OPT-IN/RECAL (model swap)
B2	Native kernels exist but wheels are non-gating , so most installs run pure Python	CONCERNS §Deps; <code>.github/workflows/ci.yml</code>	§7.1 native-default + wheels-as-gate	PARITY-SAFE
B3	Postgres <code>embedding IS NULL</code> fallback embeds many rows in Python per query	<code>postgres_engine.py</code> ; CONCERNS §Perf	§7.4 backfill + cap	APPROX-LAYER (index); PARITY-SAFE (scoring)
B17	<code>embedding_partition='none'</code> rows (the schema default) have no HNSW index at all — partial indexes cover only <code>public / private</code> ; default-partition rows silently exact-scan	<code>sql/schema.sql:42,50–51,89–90</code>	§7.4 index-coverage fix	APPROX-LAYER
B4	Local <code>vector_search / lexical_search</code> scan candidates in Python without native; <code>engine.py</code> still front-loads an O(N) per-hit embedding pass before the single <code>dense_scan</code> FFI crossing	CONCERNS §Perf; <code>engine.py:1281,1328–1344</code>	§7.1 native-default; §7.3 local indexing; §7.5 stored-vector path	PARITY-SAFE

#	Bottleneck	Evidence	Primary fix	Parity tag
B5	GIL serializes native compute; parallel channels are opt-in and off by default	pipeline.py:55–65 (MNEMOSYNE_PARALLEL_CHANNELS)	§7.1 GIL release; §7.5 parallel-by-default on capable tiers	PARITY-SAFE
B6	~~No pooling~~ (corrected) Connection pooling landed in P3 (<code>_PostgresConnectionPool</code> , <code>postgres_engine.py:121, 183</code>); prepared-statement / plan reuse remains absent	<code>postgres_engine.py</code> (no <code>prepare</code> call sites)	§7.4 prepared statements	PARITY-SAFE
B7	HNSW <code>ef_search</code> not tuned per query class; no iterative index scans under filters; no Postgres server tuning beyond <code>shared_buffers</code> / <code>maintenance_work_mem</code>	external pgvector 0.8 refs; <code>docker-compose.prod.yml:109–110</code> ; <code>infra/postgres/</code> has no conf file	§7.4 <code>ef_search</code> + iterative scans + server knobs	APPROX-LAYER / PARITY-SAFE (knobs)
B8	~~No embedding cache~~ (corrected) Hashing-embedding LRU exists (<code>text.py:96</code> , 8192 entries) and SqliteEngine has the subject-scoped A1 <code>embedding_cache</code> table (<code>sqlite_schema.py:254–266</code>); the real-model HTTP path (Local/Postgres) has no content-hash cache , and query embeddings recompute per repeated query	<code>retrieval.py</code> (no cache around <code>HttpEmbeddingProvider</code>); <code>engine.py:2868, 2903</code>	§7.8 content-hash + query-embed cache	PARITY-SAFE
B18	Consolidation roles run strictly sequentially per job (5 roles), each paying the full role-LLM round-trip; independent roles are parallelizable	<code>consolidation.py:462–519</code>	§7.6 role parallelism + batching	PARITY-SAFE
B9	19,529-line <code>cli.py</code> monolith inflates import cold-start and review cost	CONCERNS §Tech Debt	§7.7 CLI decomposition; lazy imports	PARITY-SAFE
B10	Local runtime-state atomic-replace writes are single-writer under concurrency	CONCERNS §Scaling; Wave-3 concurrency note	§7.3 runtime-state batching/Postgres state	PARITY-SAFE
B11	Consolidation re-embeds and recomputes projections without incremental scoping	<code>consolidation.py</code> ; CONCERNS	§7.6 batch embed + §7.10 incremental recompute	PARITY-SAFE
B12	Python↔native boundary crossed per-item in some paths (PyFloat boxing)	<code>algorithms.py</code> / <code>dense.rs</code> (<code>dense_scan_packed</code> exists but not universal)	§7.1 bulk APIs + packed layout	PARITY-SAFE
B13	Stateless MCP mode rebuilds engine/providers per call	<code>mcp_server.py</code> (<code>_build_tools</code> , stateless rebuild)	§7.7 warm reuse / keepalive	PARITY-SAFE
B14	Cross-encoder rerank pays a second model round-trip per query	<code>retrieval.py</code> (<code>HttpReranker</code>)	§7.2 batched rerank + shortlist width policy	PARITY-SAFE (transport); OPT-IN/RECAL (model)
B19	Embedding identity is wasteful: <code>bge-small-en-v1.5</code> emits 384 dims, random-projected to 1024 (<code>services/embedding/app.py:32–33</code>) — 2.7× storage/index/scan cost for zero added information; PostgresEngine MMR additionally re-hashes text instead of reusing stored pgvector vectors (<code>postgres_engine.py:4128</code> , opt-in fix exists)	<code>services/embedding/app.py</code> ; <code>postgres_engine.py:4128</code>	§7.2.6 model-identity decision; §7.5 stored-vector MMR	OPT-IN / RECAL
B20	Latent 10k→100k scale cliffs : local/10000 p50 already ≈ 400 ms and sqlite/10000/deep p95 ≈ 1.75 s in the committed delta report — the §22.5 budget is at risk at 10k on some cells and 100k is entirely unmeasured	<code>eval/latency/reports/perf-delta-20260705.md</code>	§7.3/§7.4 indexes; §11.8 measurement plan; native-accel P6 gate	APPROX-LAYER

6. The performance-architecture thesis

Everything below flows from one diagram: split retrieval into a **fast approximate funnel** and an **exact oracle apex**, and put all approximation, quantization, and SIMD strictly below the apex.



Three rules make this a no-compromise design rather than a tradeoff:

- The apex is sacred and cheap.** Because the shortlist N is tiny relative to the corpus, running the exact `f64` kernel over it costs microseconds. Exactness is essentially free once candidate generation has narrowed the field. We never approximate the apex, so ECE (0.0063) and the parity proof are untouched.
- The funnel is where speed lives.** ANN indexes are *already* approximate; making them faster (quantized vectors, disk-resident graphs, iterative scans, higher/lower `ef_search`) changes *which* candidates reach the apex, governed by the `recall@k/nDCG` SLO gate — not by the parity oracle. This is where quantization and SIMD-class libraries are legitimately usable.
- Caching wraps the whole thing and returns identical bytes.** A cache hit is by definition byte-identical to a miss, so caching is unconditionally PARITY-SAFE as long as it caches post-sanitization, post-redaction results keyed on the full request context (tenant, branch, sensitivity ceiling, as-of time).

This thesis is exactly why the answer to the parity-vs-speed question is “keep strict byte-parity everywhere” *and* “go fast” simultaneously: the two operate at different layers.

7. Layer-by-layer refactoring & acceleration plans

Each subsection is an independently executable work package.

7.1 Native kernel tier — realize the acceleration that already exists

Current state. `rust/mnemosyne-native/` ships bit-parity kernels for `hashing_embedding`, `tokenize`, `lexical_score / lexical_scan`, `cosine / dense_scan / dense_scan_packed`, `mmr_select_indices`, and `ppr_power_iteration`. They are scalar-sequential `f64`, rayon **across** items only, `lto=true`, `codegen-units=1`. Measured (committed micro-gates): lexical **41x**, PPR dispatch **12x**, hashing **~4.9x**, prepacked/packed-BLOB dense **≥10x** (SQLite packed path ~43x); native-vs-pure e2e at deep/local/1000: **28.3 ms vs 153.4 ms p50**. But per `.github/workflows/ci.yml` the **native wheel job is non-gating**, and `capability.py:130` maps a host with no `mnemosyne_native` module to the `floor` tier — i.e. the default install runs pure Python.

The problem in one sentence. The fastest, already-proven, already-byte-parity code path is the one most users never execute.

Work items (all PARITY-SAFE):

- Make wheels a release gate.** Promote the `native-wheels` and `test_native_parity.py` jobs to required CI status on `main` and release tags. Build `abi3-py312` wheels for the supported matrix (Linux `x86_64/aarch64` via `manylinux`, macOS `arm64/x86_64`, Windows `x86_64`) with `maturin`. Ship them on PyPI so `pip install mnemosyne-memory` gets native by default, with pure-Python as the guaranteed fallback (still canonical for parity).
- Flip the default tier when native is present.** `capability.py` already promotes a native + ≥9 GiB host to `standard`. Ensure the `standard/accelerated` tiers actually *route* retrieval through the native kernels by default (not just advise `env`). Keep `MNEMOSYNE_PURE=1` as the explicit escape hatch and the parity oracle.
- Release the GIL around native compute.** Wrap the rayon-parallel kernels in `Python::detach` (renamed from `allow_threads` in PyO3 0.29+, which the crate already targets). Per the PyO3 guide, failing to detach while rayon workers contend for the GIL can deadlock; detaching is also what lets the \$7.5 parallel channels overlap with native compute instead of serializing on the GIL. This changes *scheduling*, not *numerics* — parity holds.

4. **Kill per-item boundary crossings.** Extend the `dense_scan_packed` pattern (little-endian `f64` bytes read in place via `f64::from_le_bytes`, no PyFloat boxing) to every hot kernel entry, and pass whole candidate blocks across the boundary once per channel rather than per hit. The PyO3 guidance is explicit: even with zero-copy buffers, crossing the boundary too often dominates; design bulk APIs. Use the buffer protocol / `rust-numpy PyArray` for zero-copy where a contiguous array already exists. In particular, collapse the $O(N)$ per-hit `_embedding_for_hit` pass in `engine.py:1328-1344` into the packed stored-vector path so the dense scan does not front-load N Python calls.
5. **Contiguous columnar candidate layout.** Store/serve candidate embeddings as one contiguous `f64` (or `f32` → widened) block per channel so the scalar Neumaier loop streams cache-friendly memory. Layout changes improve cache locality without changing the arithmetic order, so the compensated-sum result is bit-identical.
6. **Extend native coverage to the remaining named seams.** The native-acceleration design names `journal`, `projections`, and `honeytokens` as seams, but today only retrieval kernels are accelerated; those three remain pure-Python greenfield. The highest-value next kernel is the **projection fold loop** (decode journal event → apply → fold state), a textbook PyO3 batch workload — see §7.10. Honeytoken scanning (`scan_for_foreign_honeytokens`) over large exports is a natural `fst`-based dictionary-scan kernel if profiling ever shows it hot; do not build it speculatively.

Why this is safe. None of these change the float-op sequence. The Neumaier core in `dense.rs` runs the identical products in the identical order; rayon parallelism is *across rows*, each row summed sequentially, exactly as the parity contract requires.

Projected gain (to prove). Native scalar kernels are measured 10–41× on their hot loops; GIL release + parallel channels add near-linear scaling across the 3 retrieval channels on multi-core hosts. Because the engine portion of the warm path is already small, the visible end-user gain is largest on the local backend and on cold/large-corpus scans.

Verification. `tests/test_native_parity.py` must stay green (byte-identical); the committed baseline gates in `tests/benchmarks/test_retrieval_baselines.py` (10×/3× exit gates against `baselines.json`) must hold; add a CI assertion that the wheel loads and `parity_marker()` returns `strict-ieee-scalar-v1`.

7.2 Embedding & rerank provider layer — attack the dominant warm cost

Current state. `HttpEmbeddingProvider` supports OpenAI-style batch (`embed_many` with `{"input": [...]}`) with per-item fallback; the reference model is `BAAI/bge-small-en-v1.5` — which natively emits **384 dims, random-projected to 1024** (`services/embedding/app.py:32-33`) — served by `services/embedding/app.py` (FastAPI/uvicorn or stdlib fallback; model lazily loaded on first request, so the first caller eats the cold start, `app.py:206`), with a `cross-encoder/ms-marco-MiniLM-L-6-v2` reranker. A Rust sidecar reimplementation exists (`rust/mneme-providers/`, `axum + fastembed`) but serializes on `Arc<Mutex<FastEmbedBackend>>` and embeds one input at a time (`lib.rs:23,242,351,382`) — no true batching — and has **zero recorded bake-off numbers**. Per §3.2 this layer is the warm-path latency.

Work items:

1. **Adopt HuggingFace Text-Embeddings-Inference (TEI) as the recommended production embedding/rerank server** (APPROX-LAYER only if the model/precision changes; otherwise PARITY-SAFE). TEI (Apache-2.0, Rust) brings **token-based dynamic batching** (batches by token count, not request count — kills padding waste), Flash-Attention/Metal/CUDA/Intel-MKL/ONNX backends, and production telemetry — and it can host the cross-encoder reranker in the same container. It slots directly behind the existing `HttpEmbeddingProvider / HttpReranker` contracts; keep the current `services/embedding` deterministic fallback as the offline/local default.
2. **Run the `mneme-providers` bake-off the native-accel program already requires** (PARITY-SAFE decision input). The sidecar is built but defaulted for lack of evidence. Bench it against `services/embedding` (fp32 torch) and TEI on the same model/hardware; fix its two known defects first if it is to compete seriously (per-call mutex serialization; single-input embed loop → add a true batch path). Whichever provider wins becomes the *recommended* deployment; the losers remain contract-conformant options (§7.11.1 unifies the contract testing).
3. **Colocate the embedder with the engine to delete HTTP/JSON overhead** (PARITY-SAFE). Offer an in-process ONNX Runtime embedding provider (via `fastembed` or ONNX-int8 artifacts; typical 2–4× CPU speedup at <1% quality loss for BERT-class embedders) as an opt-in that removes the localhost round-trip entirely for single-node deployments. Same model, same weights → same vectors → PARITY-SAFE. This directly compresses `embed_call_only`. Pin model artifacts by SHA in the evidence bundle — embedding-model identity is part of retrieval reproducibility.
4. **Dynamic request-side batching for concurrent queries** (PARITY-SAFE). Add a short (e.g. 2–5 ms) coalescing window in the provider adapter so concurrent queries share one `embed_many` call. Batching changes throughput and tail latency, not vectors. Wire `embed_many` through the retrieval hit-embedding path too — it is currently wired for consolidation (`consolidation.py:82-89`) but not the retrieve hot path.
5. **Batched cross-encoder rerank + shortlist-width policy** (PARITY-SAFE transport). Send the whole shortlist in one rerank request; make `rerank_width` an explicit `OperatingPolicy` tunable per query class (fast vs deep route already exists in `route()`), so fast-path queries rerank a narrow shortlist and deep queries widen it.
6. **(OPT-IN / RECAL) Fix the embedding-model identity.** The 384→1024 random projection (B19) inflates every stored vector, every index page, and every dense scan by 2.7× while adding zero information. Two candidate resolutions, both requiring full re-embed + recalibration + ECE re-proof, so both are OPT-IN and decided by bake-off evidence: (a) store native 384-dim (cheapest; halves-plus index cost with identical information), or (b) adopt an MRL-trained model (e.g. `nomic-embed-text-v1.5`, Apache-2.0, 64–768d Matryoshka) enabling the truncation cascade in item 7. Until decided, the 1024-dim path remains the oracle.
7. **(OPT-IN / RECAL) Matryoshka cascade.** For models trained with Matryoshka Representation Learning, shortlist on a truncated prefix (e.g. 256-dim retains ~89–97% retrieval quality across published tests) and re-rank on the full-dim vector — published results report up to ~14× wall-clock speedups for adaptive cascades at equal accuracy. Combined with `halfvec` (§7.4.5), a 256-dim fp16 index is ~6× smaller than the current 1024-dim fp32 one. Off by default, gated behind full recalibration; the full-dim path remains the oracle.

8. **(OPT-IN, additive) Tier-0 static-embedding prefilter.** `model2vec` /potion static embeddings (MIT; ~8–30 MB models; up to ~500× faster than transformer embedders on CPU, at ~92% of MiniLM quality) are microsecond-per-doc — cheap enough to run inline at ingest for (a) near-duplicate detection (skip re-embedding unchanged content), (b) coarse candidate generation as an additional APPROX funnel leg, (c) embedding fallback when the provider is down. Never a replacement for the main embedder; purely additive funnel material.

Projected gain (to prove). Colocation removes the HTTP/JSON tax from the dominant term; dynamic batching multiplies throughput under load; TEI's dynamic batching flattens the P99 tail. Combined with \$7.8 caching, warm P95 should fall well under 100 ms on repeat-heavy workloads.

Verification. Re-run `eval/latency/bench.py --clients N --queries M` against (a) current `services/embedding`, (b) TEI, (c) `mname-providers`, (d) in-process ONNX; assert identical vectors for the “same model” configs (PARITY-SAFE proof) and record P50/P95/P99 + throughput deltas. Any model/precision change re-runs `eval/calibration` and must keep $ECE \leq 0.05$.

7.3 Local in-memory & SQLite path — indexing and contention

Current state. Local `vector_search` / `lexical_search` are linear Python scans (native-accelerated when present). SQLite backend is per-tenant WAL with optional `sqlite-vec` (declared as the `sqlitevec` extra) and the packed-BLOB dense seam (~43× measured, ~9 ms retrieves). Local runtime-state persistence is single-writer via atomic-replace, which the Wave-3 note flags as a concurrency contention point. The committed delta report shows local/10000 p50 ≈ 400 ms and sqlite/10000/deep p95 ≈ 1.75 s (ambient-load caveat applies) — the 10k→100k trajectory is the concern.

Work items (all PARITY-SAFE unless noted):

1. **Ship `sqlite-vec` as the default SQLite vector index** where the extra is installed, with the linear scan as fallback (APPROX-LAYER). This turns O(corpus) local scans into indexed candidate generation while the exact kernel re-scores the shortlist (funnel/apex split, §6). The native-accel design already names the escape hatches if `sqlite-vec` stalls (usearch swap; SQLite `vec1`) — keep those as revisit triggers, not work.
2. **Add an optional in-memory ANN index for `LocalMemoryEngine`** (APPROX-LAYER) — a lightweight HNSW (e.g. embedded `usearch`, Apache-2.0, mmap-servable) built only over the tenant's live set as an APPROX candidate generator, gated by corpus size: small corpora keep the exact linear scan (already fast); large corpora get the index. Because it is a *derived, rebuildable* structure, crypto-shred correctness is preserved (rebuild after shred), and it must carry honeypot canaries like any new boundary (§4.5).
3. **Batch/relax local runtime-state writes** (B10): coalesce metric persistence (write-behind buffer flushed on interval or on N updates), or route to a per-tenant serialized writer, so concurrent searches stop contending on atomic-replace. In production, prefer Postgres runtime state. This is the fix for the single-writer benchmark artifact.
4. **Memory-map large per-tenant SQLite reads** and set pragmatic WAL/ `mmap_size` / `cache_size` pragmas for the read-heavy retrieval path.

7.4 Postgres path — candidate generation at scale

Current state. Production schema is `VECTOR(1024)` with HNSW (`vector_cosine_ops`) on `evidence.embedding` and `assertions.embedding` — but **only as partial indexes over `embedding_partition IN ('public', 'private')`**, while the column default is `'none'` (`sql/schema.sql:42,50–51,89–90`): default-partition rows have no vector index and exact-scan. Tenant RLS, FTS (stored `lexeme` TSVECTOR + GIN — landed in P3), recursive PPR with the persisted `graph_ppr_cache`. Connection pooling landed in P3 (`postgres_engine.py:121,183`); prepared statements did not. The `embedding IS NULL` fallback embeds rows in Python per query (B3). Server tuning is minimal: `shared_buffers=512MB` + `maintenance_work_mem=512MB` on the compose command line; no `work_mem`, `effective_cache_size`, `max_parallel_workers`, or `hnsw.ef_search` anywhere (`infra/postgres/` contains no tuning file).

Work items:

1. **Backfill null embeddings during ingestion/consolidation and cap the fallback** (B3; APPROX-LAYER for the index, PARITY-SAFE for scoring). Make embedding at write time the contract so the read path never runs the unbounded per-query Python embed loop. Add a null-embedding counter to observability; cap/paginate any residual fallback scan; treat a nonzero null-embedding count in production as an alert, not a silent slow path.
2. **Close the `embedding_partition='none'` index gap** (B17; APPROX-LAYER). Either (a) add a third partial HNSW index covering `'none'`, (b) make the ingest path always resolve a concrete partition so `'none'` cannot persist (then constrain it), or (c) replace the partial-index scheme per §7.11.4. Whichever lands, add a drift check asserting every row is covered by exactly one vector index — silent exact-scan is the failure mode this blueprint exists to kill.
3. **Prepared statements + plan reuse on the pooled connections** (B6-residual; PARITY-SAFE). The pool exists; add prepared-statement reuse for the hot retrieve/append statements (psycopg3 `prepare_threshold` or explicit `PREPARE`), and verify the pool's session settings pin `hnsw.ef_search` per §7.4.4.
4. **Per-query-class `hnsw.ef_search`** (B7; APPROX-LAYER). Set `ef_search` as a session GUC keyed to the fast/deep route: ~20–40 for fast-path recall/speed, ~100–200 for deep/high-accuracy. This is the single most direct recall/latency dial on HNSW; raising it improves recall at ~linear latency cost, so tie it to the route heuristic already in `engine.py::route()`.
5. **pgvector 0.8 iterative index scans** (APPROX-LAYER). Enable `hnsw.iterative_scan` so RLS/sensitivity-filtered queries keep recall high without over-scanning; use `strict_order` where the funnel must preserve exact distance ordering into the apex, `relaxed_order` where the apex re-score will re-order anyway. Multi-tenant queries are *always* filtered (tenant_id RLS + sensitivity ceilings) — this is precisely the workload iterative scans were built for.
6. **halfvec HNSW index for candidate generation + exact f64 rescoring** (APPROX-LAYER for the index; PARITY-SAFE for returned scores). Store a 2-byte `halfvec` copy for the index (halves index footprint, near-identical recall at `ef_construction=256`, identical-or-slightly-better

query time per pgvector maintainers), generate candidates from it, then re-score the shortlist with the exact `f64` cosine kernel from the full-precision vectors. The oracle never sees the half precision.

7. **Postgres server tuning file** (PARITY-SAFE). Add a version-controlled `infra/postgres/postgresql-perf.conf` (mounted via compose `-c config_file` or appended `-c flags`) setting at minimum: `work_mem` (per-sort/hash memory for FTS + rerank joins), `effective_cache_size` (planner honesty on the resized VM), `max_parallel_workers_per_gather` (parallel seq/bitmap scans for analytic jobs, not the hot path), and `jit=off` for short OLTP-style queries. Pair every knob with a before/after bench row; register in `CONFIG-DRIFT-CHECKS.md`.
8. **(Large tenants) pgvector scale StreamingDiskANN** (APPROX-LAYER). For tenants whose vector set exceeds RAM-resident HNSW economics, offer `pgvector scale StreamingDiskANN` (PostgreSQL-licensed, Rust) — disk-resident StreamingDiskANN with SBQ quantization and streaming post-filtering that provably does not miss filtered results; published ~28× lower p95 at 99% recall vs unturned pgvector HNSW at 50M vectors. Wire it through the existing adapter pattern so engine code is untouched; keep pgvector HNSW as the default. (VectorChord posts stronger numbers still, but its AGPLv3/ELv2 dual license fails the \$9.1 governance bar for a default; its RaBitQ *algorithm* is separately adoptable — \$9.1.)
9. **Index-only scans and partitioning** (PARITY-SAFE). Ensure retrieval projections are covered by indexes to enable index-only scans; consider per-tenant (or tenant-hash) partitioning for the largest tables to bound scan sets and vacuum cost — with the bonus that **a shredded tenant's partition can eventually be DROP ped, making physical erasure trivially provable** (§7.10.5). Keep RLS semantics identical.
10. **Prepared/cached PPR and graph-signal reuse** (PARITY-SAFE). The recursive PPR is the heaviest SQL; per the settled OQ1 decision, cached PPR serves the fast path and live PPR runs async-deep only. Verify the B2 pair-index memo covers all call sites (`engine.py:1482` was the O(V-E) scan), reuse `graph_ppr_cache` per tenant/branch/as-of with correct invalidation on write, and only revisit the PPR design at the OQ1 tripwire (~10⁶ edges/tenant or fast-path P95 breach).

Verification. Live Postgres lanes (`tests/test_postgres_engine_live.py`, `tests/test_postgres_perf_lanes.py`) with DSN armed; assert recall@k/nDCG hold under each APPROX change; assert exact-rescore path returns byte-identical scores to the full-precision kernel on a fixed shortlist; new drift check for total vector-index coverage (item 2).

7.5 Retrieval pipeline orchestration — overlap and reuse

Current state. `run_retrieval_pipeline()` runs dense/lexical/graph channels; parallelism is opt-in via `MNEMOSYNE_PARALLEL_CHANNELS` (`ThreadPoolExecutor(max_workers=3)`, `pipeline.py:55–65,146`), default off. RRF/MMR are in `algorithms.py` (native-mirrored; RRF shallow-reconstruction already landed in P3, killing the `copy.deepcopy` tax). PostgresEngine MMR re-hashes text into hashing-space instead of reusing stored pgvector vectors; the stored-vector MMR fix exists but is gated opt-in (`postgres_engine.py:4128`, commit `b8347b7`).

Work items (all PARITY-SAFE unless noted):

1. **Enable parallel channels by default on standard / accelerated tiers** (B5). With the GIL released around native compute (§7.1) and I/O-bound SQL/HTTP channels, overlapping the three channels is a strict latency win. `capability.py` already *recommends* `MNEMOSYNE_PARALLEL_CHANNELS=1` for accelerated tiers — make it the resolved default there, keep it off on `floor`.
2. **Overlap embedding with lexical/graph** — fire the query embed and the lexical/graph channels concurrently; fuse when the dense channel returns. Fusion order is deterministic (RRF over ranked lists), so concurrency does not change results.
3. **Structured async I/O for provider + SQL round-trips** — where multiple network/SQL calls are independent, issue them concurrently with bounded concurrency (4–8 for I/O; 1 for CPU-bound native compute or shared-state writers).
4. **Deterministic fusion under concurrency** — assert RRF/MMR consume channel outputs in a fixed canonical order regardless of completion order, so parallelism is provably parity-safe (add a test that shuffles completion order and asserts identical fused output).
5. **(OPT-IN / RECAL) Default the stored-vector MMR path.** Reusing stored pgvector vectors for MMR instead of re-hashing (B19 second half) is both a perf win (no per-candidate re-embed) and a *quality* win (MMR diversifies in the real embedding space, not hashing space) — but it changes ranking numerics, so it follows the OPT-IN ladder: bake-off, recalibration, ECE re-proof, then default-flip proposal.

7.6 Consolidation & background jobs — batch, parallelize, incrementalize

Current state. `consolidation.py` (3,225 lines) runs the warm-loop role pipeline; the five roles execute **strictly sequentially per job** (`consolidation.py:462–519`), each paying the full role-LLM round-trip — today 95–320 s per call on the un-flipped runtime (B15). Batch embedding is wired for consolidation (`consolidation.py:82–89`); `MNEMOSYNE_EMBED_BATCH_SIZE` exists (tier-tuned in `capability.py`).

Work items (all PARITY-SAFE):

1. **Parallelize the independent roles** (B18). Summarizer/lesson-distiller/procedure-inducer roles that do not read each other's output within a pass can run concurrently; preserve the documented order for dependent roles (`extractor` → `resolver` → `belief_reviser` → `gate`). The win multiplies with Wave 0: 5 sequential × 2–5 s becomes ~max(2–5 s) for the independent set. Role ordering and mutation-rail budget accounting interact — pin the current behavior with fixture tests before touching the loop.
2. **Batch all consolidation embeddings** through `embed_many` with the tier-tuned batch size (8/32/64/128 by tier), replacing any residual per-item embed in the pipeline. Batching changes throughput, not vectors.
3. **Incremental projection recompute** — scope recompute to the affected set per the settled OQ3 decision (salsa-style memoization on the existing queue: `input_fingerprint` + red/green dirty-check; **no** `pg_ivm`, **no** differential-dataflow) rather than full re-derivation. §7.10 carries the deeper projection-rebuild machinery.
4. **Bounded worker concurrency** — allow N consolidation workers with per-tenant leasing (the queue already has lease semantics) so throughput scales while the §31 cadence rail (R7) and mutation-rail budget stay enforced per tenant.

5. **Sleep-time scheduling** (design steal from Letta, §9.2). Consolidation, lifecycle sweeps, projection compaction, and cache prewarming are all deferrable work — schedule them into idle windows (the job queue already carries `calibrate / lifecycle-sweep / observability-snapshot` job kinds) so hot-path load never competes with background cognify. Purely a scheduling policy; every consolidation remains journaled and gated exactly as today.

7.7 CLI/MCP surface & cold start — decompose the monolith

Current state. `cli.py` is **19,529 lines** — command router, runtime loader, ops checker, deployment-soak runner, release-audit validator, and production-evidence verifier in one file (CONCERNS top tech-debt item; ~⅓ of the entire package). `mcp_server.py` supports a stateless mode that rebuilds tools per call (B13); its hosted transport is a synchronous stdlib `ThreadingHTTPServer` (`mcp_server.py:14`) with the MCP SDK `StreamableHTTP` edge living in the separate `mnemo-stream` container. MCP cold start already improved 112.6→88.3 ms self-time via P3 lazy imports; the residual is mostly stdlib import weight.

Work items (all PARITY-SAFE; structural):

1. **Decompose `cli.py` behind the stable `argparse` facade.** Extract cohesive, independently-testable modules — `cli_ops/` (provider-check, deployment-soak, release-audit, `*-ops-check`), `cli_evidence/` (capture/verify/manifest), `cli_runtime/` (engine/provider/queue wiring) — leaving `cli.py` as a thin parser+dispatch. Per CONCERNS, do this **seam by seam, each pinned by a focused test**, not as a big-bang rewrite. This cuts review cost, reduces the blast radius of command changes, and shrinks import cold-start (only the subcommand's module loads).
2. **Deepen lazy imports** (already used heavily) so the CLI/MCP cold path imports only what a given command needs — directly attacks the interpreter/import cold-start that still affects real one-shot CLI invocations.
3. **Warm MCP reuse / keepalive** (B13) — avoid per-call engine/provider rebuild in stateless mode by caching an immutable engine handle keyed on config, or documenting stateful mode as the throughput default. The build cost (engine+corpus ≈ 1.75 s warm-once in the Postgres bench) must be paid once per server, never per request.
4. **Hold the Rust front-end gate.** The native-accel Phase-4 question (“Rust `rmcp` front-end / warm daemon?”) stays governed by its own data gate: the captured MCP trace shows ~88–116 ms stdio init, which does **not** currently justify a second front-end implementation. Decide with the Phase-4 spawn-latency evidence — “open or permanently close with data, not vibes” — and record the outcome as an ADR either way.
5. **Optional: compile hot pure-Python helpers.** For any residual Python hot loop not worth a Rust kernel, evaluate a `mypyc` /Cython build of the specific module (PARITY-SAFE if it preserves float semantics; verify against the parity suite). Prefer moving genuinely hot math to the existing native crate over introducing a second compiled toolchain.

7.8 Caching strategy — multi-tier, byte-identical

Current state (corrected in v1.1). Three cache surfaces already exist: the hashing-embedding LRU (`text.py:96`, 8192 entries, backend-agnostic by design), the SQLiteEngine subject-scoped A1 `embedding_cache` table (`sqlite_schema.py:254–266`, privacy-aware admit/purge logic in `sqlite_engine.py`), and `GraphSignalCache` (per tenant/branch graph hits) plus the persisted `graph_ppr_cache` table. **Missing:** any cache in front of the real-model HTTP embedding path used by Local/Postgres, any result cache, and any calibration-lookup cache.

Work items (all PARITY-SAFE — a hit returns identical bytes to a miss):

1. **Content-hash real-model embedding cache** keyed on `(model_id, model_revision, sha256(normalized_text))`, fronting `HttpEmbeddingProvider` for both query and document embeddings. Embedding is deterministic for a fixed model+revision, so this is exact. It removes the dominant warm cost (B1/B8) for repeated and templated queries, and turns deliberate re-projection passes from $O(\text{embed})$ into $O(\text{hash lookup})$. Respect the same subject-scoped admit/purge policy the A1 SQLite cache already implements (privacy tightening purges the row); seed honeytokens into the cache namespace (§4.5).
2. **Result cache** keyed on the **full** request context: `(tenant, branch, query, k, route, sensitivity_ceiling, as_of, policy_version, calibration_version)`. Must cache the **post-sanitization, post-redaction** `RetrievalResult` so §31 R6 and access-policy boundaries are never bypassed. Invalidate on any write to the tenant/branch (evidence append, supersession, forget) and on policy/calibration version bumps.
3. **Calibration lookup cache** — conformal calibration parameters per tenant/memory-type are read on every scored answer; cache the resolved calibrator with invalidation on `conformal_calibration` updates.
4. **Negative/abstention cache** — cache calibrated abstentions too, so “I don’t know” is as fast as an answer and doesn’t re-run the funnel.
5. **Cache-safety invariants** — a dedicated test proves cache hits are byte-identical to misses across the sensitivity/as-of/branch matrix, that a write correctly invalidates, and that no foreign honeytokens ever surfaces from any cache tier. This is what makes caching provably PARITY-SAFE rather than merely fast.
6. **(OPT-IN, runtime-level) KV-cache activation prewarm.** Since role/parametric LLM serving is local (Ollama on host Metal after Wave 0), pre-warming the model’s KV cache with a tenant’s stable prompt prefix (core profile projection, §9.2.1) is a real TTFT win that no Postgres-side change can match (MemOS reports ~90%-class TTFT cuts from served activation memory). This is a *runtime* cache below the engine: it never touches retrieval numerics. Gate on measured TTFT evidence; account for VRAM/RAM per pinned prefix.

7.9 Production runtime & database server tuning — execute the committed flips (new in v1.1)

Current state. §3.5 in full: host-Metal LLM topology and VM right-sizing are committed, guarded, and **dormant**; the evidence-capture precondition cleared 2026-07-07. Postgres runs with two memory knobs and no server tuning file (B7).

Work items (all PARITY-SAFE — configuration and topology only; zero engine-code change):

1. **Wave 0 flip (operator action).** With operator confirmation: run `infra/scripts/apply-perf-runtime.sh` under `MNEMO_CONFIRM=1` — colima resize to 6 vCPU/12 GiB (full stop/start), `docker compose up -d` to apply the committed per-service ceilings and the host-Metal default topology. Expect Vault to come back **sealed** (manual unseal via `vault.mnemo.local`; known crash-loop of `mnemo-api / stream / parametric` until unsealed — auto-recovers after). Verify with the stack health checks and a single consolidation role call timed before/after.
2. **Post-flip re-baseline (blocking before any engine-win claims).** Re-run the P0 harnesses (`eval/latency/perf_e2e_bench.py`, `eval/latency_warm/bench_warm.py`, MCP trace) on the resized runtime under a **quiesced-host protocol** (record load average; reject runs above a documented ceiling) — the 2026-07-05 delta run's ambient-load contamination must not recur. Commit the new baselines; every subsequent wave measures against these.
3. **Consolidation wall-clock capture.** The P0 spec's per-role wall-clock line item has no committed result file; capture it on the flipped runtime (it is the direct evidence for §7.6.1's parallelization win).
4. **Postgres server tuning** per §7.4.7, sized against the *resized* VM (`effective_cache_size` in particular is meaningless before the resize).
5. **Rollback discipline.** `rollback-perf-runtime.sh` is the paired escape hatch; the flip is reversible and must stay so. Document the observed flip in `infra/PRODUCTION-EVIDENCE.md` with before/after numbers — projected 48–55× on role calls becomes a measured figure or it didn't happen (§254).

7.10 Journal & projection performance package (new in v1.1)

Current state. The per-tenant CID journal (`journal.py`, 202 lines) and the projection registry (`projections.py`, 77 lines) are pure Python — named native seams with no kernels. Projection recompute runs on the queue; the settled OQ3 decision mandates salsa-style memoization (`input_fingerprint`, red/green dirty-check) and forbids `pg_ivm`/differential-dataflow. Event-sourcing performance engineering (snapshotting, group commit, blue/green rebuilds) is not yet systematized. This package adapts the proven patterns from mature Postgres event stores (Marten's async-projection/snapshot machinery; the eugene-khyst PostgreSQL-event-sourcing analysis) to Mnemosyne's ledger-first/journal-second design.

Work items:

1. **Projection snapshots (PARITY-SAFE).** Persist projection state every N events (or bytes/time threshold) as *disposable derived data*, version-stamped by the projection-code fingerprint the registry already computes — a fingerprint change auto-invalidates the snapshot. Rehydrate = latest valid snapshot + tail replay. Rule of thumb from production event stores: only snapshot streams past a few hundred events; then it is a 10–100× rehydration win.
2. **Blue/green validated projection rebuilds (PARITY-SAFE).** Rebuild into `projection_v2` alongside the live table, run the validator suite against it (Mnemosyne's validator culture slots in exactly here), then swap via rename/view flip. Zero downtime, and the swap itself becomes an evidence artifact. During rebuild: fold in memory and write once per entity (not row-by-row), `synchronous_commit=off` **for the rebuild session only** (projections are rebuildable; the journal never relaxes durability), drop/re-create indexes around the bulk load, write via `COPY` /multi-row inserts.
3. **Rust projection-fold kernel (PARITY-SAFE; the highest-value new native seam, §7.1.6).** The fold loop — decode canonical-JSON event → apply → fold state — is a textbook PyO3 batch workload; pair it with a `COPY` writer for rebuilds. Expect 5–20× rebuild speedups (projected; prove on the bench seam). CID computation stays in Python per the native-accel contract — the kernel folds *already-journaled* events and never mints identity.
4. **Micro-batched appends + group commit (PARITY-SAFE).** Application-level micro-batching (accumulate journal appends 5–20 ms under load, single multi-row insert) keeps p99 predictable and amortizes fsync; Postgres `commit_delay` / `commit_siblings` add 2–5× commit throughput under concurrency. The journal's append-only semantics and CID chaining are unchanged — batching is a transport arrangement, and the ledger-first/journal-second ordering rule is preserved within each batch.
5. **Per-tenant partitioning with crypto-shred synergy (PARITY-SAFE).** Partition the evidence/journal-adjacent tables by tenant (or tenant-hash): projection rebuilds parallelize by partition, vacuum cost is bounded, and a crypto-shredded tenant's partition can eventually be **DROP** ped — turning “the key is destroyed and the ciphertext is unreachable” into “the bytes are physically gone,” a strictly stronger erasure story with no policy change.
6. **Gap-safe checkpointed consumers (PARITY-SAFE; correctness-critical).** Each projection/queue consumer stores its last-processed position; batches claimed via `FOR UPDATE SKIP LOCKED`; `LISTEN/NOTIFY` used only as a doorbell (payloads are not durable). Postgres sequences are neither gap-free nor commit-ordered — adopt a transactional sequencer (single-writer ticket or advisory-lock assignment) or explicit gap-detection-and-wait before any consumer assumes ordering. This is the #1 latent-correctness bug class in naive Postgres event stores; close it while instrumenting the path.

Verification. Property tests already cover journal invariants (`test_journal_properties`); add rebuild-equivalence tests (blue/green output byte-identical to a from-scratch fold), snapshot-invalidation tests (fingerprint bump forces full replay), and a bench-seam lane for fold throughput (events/s, pure vs native).

7.11 Structural architecture refactor register (new in v1.1)

Beyond `cli.py` (§7.7), the architecture audit surfaced five structural debts. None change runtime behavior; all reduce drift risk and review cost. All PARITY-SAFE / structural.

1. **Unify the provider contract (three implementations, two languages).** The `/embed` + `/rerank` + `/health` HTTP contract is implemented three times: `services/embedding/app.py` (Python real + deterministic fallback), `rust/mnemo-providers` (axum/fastembed), and the in-process deterministic fallbacks in `retrieval.py` — with TEI arriving as a fourth conformer (§7.2.1). Extract a **single versioned contract spec**

(request/response schemas, normalization, zero-vector rejection, dims/truncation semantics, auth) plus one language-agnostic **conformance test suite** (golden requests/responses + property checks) that every provider must pass in CI. Then *declare* a primary per deployment profile (bake-off-decided) instead of maintaining three first-class implementations. Contract drift across three codebases is a silent-quality-bug factory; this converts it into a red CI lane.

2. **Single source of schema truth.** Postgres schema currently evolves in two places: `sql/schema.sql` (idempotent CREATEs) and in-engine drift-repair (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS` , legacy-index drops — `postgres_engine.py:578,650+`). Adopt ordered, versioned migration files (plain SQL under `sql/migrations/` , applied by a tiny runner or the existing ensure-step machinery) and **freeze the in-engine DDL** to a verifier that *detects* drift and refuses, rather than silently repairing. Hot paths already stopped issuing DDL in P3 (`bd1409b`); this completes the separation. New DDL from this blueprint (halfvec columns, partition coverage, partitioning) lands as migrations from day one.
3. **Pipeline-Protocol guardrail.** `pipeline.py` exists precisely because Local and Postgres once carried two near-identical ~130-line `retrieve()` bodies. Add a CI structural check (AST-level or a targeted test) asserting all three engines route retrieval through `run_retrieval_pipeline / RetrievalPipelineOps` — so a fourth copy cannot regrow silently in `SqliteEngine` or any future engine (e.g. the deferred `TursoEngine`).
4. **Decouple the index layer from the access-policy vocabulary.** HNSW indexes are physically partitioned by `embedding_partition` (public/private) — correct for security, but it couples storage DDL to the policy model: a new sensitivity partition forces an index migration, and the `'none'` default already fell through the crack (B17). Evaluate replacing partition-partial indexes with (a) a single covering HNSW index + iterative-scan filtering (§7.4.5), or (b) a generated column that *guarantees* total coverage — decide on measured recall/latency under RLS, and record as an ADR. The access-policy semantics themselves do not change.
5. **Scope decision for the shadow-cognitive tier.** ~4k LOC of explicitly shadow-only modules (`workspace` , `consciousness` , `dreamer` , `self_optimization` , `credentials` , `standing`) carry real parity-suite and review weight while gated off the production path. This blueprint takes no position on their research value — it asks for an explicit, recorded decision (keep-and-maintain / freeze-behind-feature-flag / extract-to-companion-package) so their maintenance cost is a choice rather than an accident. Route through the decision-memo pattern (§4.6).

8. Language & method strategy

The “different languages / different methods” question, answered precisely.

8.1 Where Rust wins (and stays byte-parity)

CPU-bound, scan-heavy, tight-loop math: dense/lexical scan, MMR, PPR power iteration, hashing embed, tokenization — already in `rust/mnemosyne-native/` and to become the default tier (§7.1) — plus the new projection-fold kernel (§7.10.3). Rust here buys a measured 10–41× over Python *and* holds parity because the kernels reproduce CPython’s exact float semantics. **Extend, do not replace** this crate for any new hot math. The Rust provider sidecar (`rust/mneme-providers/` , Axum + optional `fastembed`) is the right home for a colocated ONNX embedder (§7.2) *if it wins its bake-off*. For lexical/dictionary workloads that ever go hot (honeypot scans, alias matching), `fst` -based memory-mapped term dictionaries are the tool of choice (MIT/Unlicense); embedded `tantivy` (MIT) remains available strictly as a *derived, rebuildable* accelerator — never a second source of truth (§9.1).

8.2 Where Python stays

Orchestration, policy, security/redaction, ingestion, CLI/MCP surfaces, consolidation control flow. These are I/O- and logic-bound, not compute-bound; rewriting them buys little and costs the local-first simplicity and readability that are project values. Keep them Python; make them *thin* (the architecture doc’s explicit rule) and lazily imported. CID canonicalization stays in Python permanently (native-accel contract) — identity is never minted in an accelerated path.

8.3 Where SQL push-down wins

Candidate generation at scale belongs in Postgres: HNSW/halfvec/DiskANN, FTS, recursive PPR, index-only scans, partitioning. Push filtering and top-N to the database; pull only the shortlist into Python/Rust for exact apex scoring. Never pull large row sets into Python to score (the B3 anti-pattern).

8.4 Free-threaded CPython (forward-looking)

PyO3 0.23+ supports free-threaded CPython 3.13t and 0.29+ solidifies 3.14/3.14t. Free-threaded Python removes the GIL entirely, which would let the parallel channels and native kernels scale across cores without the detach dance. **Track as a future tier**, not a near-term dependency: validate the native crate under `Sync` requirements, keep abi3-py312 wheels canonical, and add a free-threaded CI lane before advertising it. This is the long-horizon ceiling-raiser for the local path.

8.5 When may byte-parity be relaxed? — Never, for the oracle.

The rule is absolute: **the deterministic scoring/fusion/calibration kernels never change numerics**. SIMD (SimSIMD ~20–200× cosine speedups), FMA, BLAS cosine, and quantized *stored* embeddings are powerful but perturb the last ULP or the stored vector, so they are confined

to the APPROX candidate-generation layer (where the recall SLO governs) or to explicit OPT-IN/RECAL tiers (off by default, gated behind full recalibration). This is the precise mechanism by which “keep strict byte-parity everywhere” and “massively faster” coexist.

9. GitHub / SOTA inspiration → specific Mnemosyne changes

Each row: an external project or technique, the exact technique to take, where it plugs into Mnemosyne, its parity verdict, and the projected gain. Take the idea; improve it by fitting it to the funnel/apex split so it never touches the oracle.

Source (license)	Technique to adopt	Plug-in point	Parity verdict	Projected gain
HuggingFace Text-Embeddings-Inference (Apache-2.0)	Rust server, token-based dynamic batching, Flash-Attention/ONNX/Metal/CUDA backends, hosts rerankers too, telemetry	Behind <code>HttpEmbeddingProvider / HttpReranker</code> ; production embedder candidate	PARITY-SAFE (same model)	Flattens P95/P99 tail; higher throughput under concurrency
fastembed (Apache-2.0)	In-process ONNX Runtime embeddings, pre-quantized models, intelligent batching	In-process provider; colocate with engine (removes HTTP round-trip); already inside <code>mneme-providers</code>	PARITY-SAFE (same model)	Deletes <code>embed_call_only</code> HTTP/JSON term
pgvector 0.8 (PostgreSQL)	<code>hnsw.iterative_scan</code> (strict/relaxed) for filtered queries; <code>halfvec</code> index; fixed cost estimation	Postgres candidate generation (§7.4)	APPROX-LAYER	Keeps recall under RLS/sensitivity filters; halves index size
pgvectorscale / StreamingDiskANN (PostgreSQL)	Disk-resident DiskANN, SBQ quantization, streaming post-filter that provably keeps filtered recall; ~28× lower p95 at 99% recall vs untuned HNSW (50M vectors, vendor bench)	Optional large-tenant index via adapter (§7.4.8)	APPROX-LAYER	Scales past RAM; big p95 win on large tenants
VectorChord (△ AGPLv3/ELv2)	Algorithm only: RaBitQ binary-quantize-then-rerank (<1% recall loss at 8-bit; 100M×768d in 32 GB @ p50 35 ms vendor bench)	Rust-seam binary-sketch prefilter (§9.1) — <i>extension itself not adopted</i>	APPROX-LAYER	Massive candidate-scan compression, license-clean when reimplemented
Matryoshka Representation Learning (technique; <code>nomio-embed-text-v1.5</code> Apache-2.0)	Truncated-prefix shortlist + full-dim re-rank cascade (~14× reported); 256d retains ~89–97% quality	OPT-IN embedding cascade (§7.2.7); model-identity fix (§7.2.6)	OPT-IN / RECAL	~6× index shrink with halfvec; large-tenant shortlist speedup
model2vec / potion (MIT)	Static-embedding distillation: ~500× CPU speedup at ~92% MiniLM quality, 8–30 MB models	Tier-0 prefilter: ingest near-dup detection, coarse funnel leg, provider-down fallback (§7.2.8)	APPROX-LAYER (additive leg)	Microsecond-class inline vectors; skips redundant re-embeds
SimSIMD / usearch (Apache-2.0)	SIMD f16/i8/b1 distance kernels (20–200× cosine); mmap-served embedded HNSW	APPROX pre-filter and local ANN <i>only</i> (never the oracle) (§7.3.2)	APPROX-LAYER (excluded from apex)	Faster candidate pre-ranking; in-process index with no new container
PyO3 guide (Apache-2.0/MIT)	<code>Python::detach</code> GIL release; <code>buffer-protocol/ rust-numpy</code> zero-copy; bulk boundary APIs	Native kernel wrappers (§7.1)	PARITY-SAFE	Parallel scaling + fewer boundary crossings
psycpg3 / PgBouncer	Prepared statements on pooled sessions; transaction pooling for QPS fan-out	Postgres engine connection layer (§7.4.3)	PARITY-SAFE	Removes re-plan overhead (pool itself already landed in P3)
Marten / postgresql-event-sourcing (MIT / Apache-2.0)	Snapshotting, async checkpointed projections, blue/green rebuilds, gap-safe sequencing	Journal & projection package (§7.10)	PARITY-SAFE	10–100× rehydration; zero-downtime validated rebuilds
Zep/Graphiti (Apache-2.0)	Bitemporal fact-validity + no-LLM-in-retrieval discipline (P95 ≈ 300 ms); hybrid fusion incl. recency	Confirms Mnemosyne’s design; recency-aware fusion leg (§9.2.4)	PARITY-SAFE (design); OPT-IN (new leg)	Validates read-path strategy; temporal-question quality
Mem0 (Apache-2.0)	Write-time ADD/UPDATE/DELETE/NOOP verdict loop; ~90% token savings vs full-context (vendor bench)	Consolidation gate event vocabulary (§9.2.3)	PARITY-SAFE (design)	Cheaper, auditable consolidation passes

Source (license)	Technique to adopt	Plug-in point	Parity verdict	Projected gain
Letta (MemGPT) (Apache-2.0)	Labeled, size-bounded core-memory blocks; sleep-time compute	Profile projection (§9.2.1); job scheduling (§7.6.5)	PARITY-SAFE (design)	Sub-ms head-serving; idle-time cognify
Memobase (Apache-2.0)	Buffer → fixed-cost batched profile merge; profile reads are pure SQL <100 ms (their LoCoMo temporal 85.05, best in table)	Profile projection + cognify buffers (§9.2.1–9.2.2)	PARITY-SAFE (design)	Bounded local-LLM load; instant head reads
HippoRAG 2 (MIT)	PPR-seeded associative retrieval with passage nodes linked into the graph (+7% associativity over best dense, no factual-QA loss)	Graph channel already PPR-based; add passage-node linking to the graph projection (§9.2.5)	APPROX-LAYER (channel input)	Multi-hop recall without triple-only lossiness
A-MEM (MIT)	Zettelkasten notes + LLM-decided links + memory evolution (retroactive neighbor re-annotation)	Consolidation emits <code>MemoryAnnotationRevised</code> events; re-projection (§9.2.6)	PARITY-SAFE (event-sourced by construction)	Self-organizing memory graph, fully audited
MemOS (Apache-2.0)	Explicit memory lifecycle states; KV-cache activation memory (~90%-class TTFT cuts, vendor bench)	Lifecycle already exists; KV prewarm at the runtime tier (§7.8.6)	PARITY-SAFE (below engine)	TTFT collapse for role/parametric calls
Free-threaded CPython 3.13t/3.14t + PyO3 0.23+/0.29+	GIL-free parallelism	Future local tier (§8.4)	PARITY-SAFE (scheduling)	Removes GIL ceiling on parallel channels

9.1 License governance & the derived-structure rule (new in v1.1)

Two standing rules keep “take inspiration from GitHub” compatible with Mnemosyne’s self-hosted, audit-grade posture:

- License gate.** Default-adoptable: PostgreSQL/Apache-2.0/MIT/BSD (pgvector, pgvector scale, TEI, fastembed, usearch, LanceDB, model2vec, simsimd, tantivy, fst, all listed memory systems). **Deliberate-decision-required:** AGPLv3/ELv2 (VectorChord, ParadeDB pg_search) — for an internally self-hosted deployment AGPL is legally workable, but it complicates any future distribution and the audit story; the blueprint’s position is **adopt the algorithms (RaBitQ-style binary-quantize-then-rerank), not the extensions**, unless a measured gap forces the question through an ADR.
- Derived-structure rule.** Any non-Postgres index or store introduced for speed (usearch/LanceDB in-process ANN, embedded tantivy, binary-sketch tables, snapshots, caches) must be **derived, rebuildable, and disposable** — rebuilt from the ledger/projections after crypto-shred, carrying honeypot canaries, never a second source of truth, never part of the transaction boundary. Networked vector-DB sidecars (Qdrant et al.) are **rejected** for the system of record on these grounds: dual-write consistency, weaker payload-filter tenancy vs RLS, and unprovable segment-compaction deletes versus Mnemosyne’s provable crypto-shred. Postgres remains the single transactional home.

9.2 Memory-capability upgrades — quality × speed steals (new in v1.1)

The agent-memory survey’s biggest lesson is convergence: every serious 2025–2026 system landed on *write-time consolidation with explicit conflict ops + hybrid retrieval fusion + background/batch cognify + pre-computed head serving*. Mnemosyne already has the first two in stronger, audited form (gated consolidation with TMS/AGM belief revision; three-channel RRF/MMR fusion). The genuine deltas — each additive, default-off, journal-native:

- Core-profile projection (Letta blocks × Memobase profiles).** A per-tenant, size-bounded, labeled projection (“who this tenant is; standing preferences; active projects; procedural rules”) rebuilt from journal events by the consolidation gate, served **without any retrieval pipeline** — a pure projection read, sub-ms from Postgres/SQLite. Injected as the always-present core context; the funnel serves the long tail. This is the pattern OpenAI’s and Anthropic’s production memory converge on (pre-compute in background, inject cheaply, keep it inspectable) — and Mnemosyne’s version is superior on provenance: every profile line traces to journal events. Redaction/ceiling rules apply at projection-build time (derived ≥ max(sources) — the S-tier laundering rule already covers this).
- Cognify buffers (Memobase).** Raw episodes append to the journal instantly (cheap, durable); a per-tenant buffer accumulates; consolidation runs as a **fixed-cost batched merge** per window instead of per-message LLM work. On a local-LLM budget this converts unbounded synchronous cognify into predictable batch load — and pairs exactly with §7.6.5 sleep-time scheduling.
- Verdict-event vocabulary (mem0).** Name the consolidation gate’s outcomes as explicit typed decisions — ADD / SUPERSEDE / INVALIDATE / NOOP — recorded on the journal. Mnemosyne already *does* the semantics (upsert-with-supersession, contradiction handling); making the verdict a first-class event improves audit legibility and enables mem0-style dedup metrics (NOOP-rate as a consolidation-quality signal) at zero behavioral change.
- Recency-aware fusion leg (Graphiti).** Add a fourth ranked list to RRF — candidates ordered by `valid_from` recency — so temporally-anchored queries (“what’s the latest...”) stop relying on dense/lexical proxies. OPT-IN / RECAL: it changes fused rankings, so it ships default-off behind the eval gate (Memobase’s temporal-question dominance suggests the win is real; prove it on the golden scenarios).
- Passage-node graph linking (HippoRAG 2).** Link evidence chunks into the relation graph as passage nodes (not just entity/assertion nodes) so PPR can land on *text*, not only on structured facts — avoiding triple-only lossiness on associative queries. APPROX-LAYER: it changes the graph channel’s candidate pool only.

6. **Memory evolution as re-projection (A-MEM).** Allow consolidation to emit `MemoryAnnotationRevised` events that retroactively re-annotate neighbor memories (keywords, links, context lines) — the Zettelkasten “network reorganizes itself” effect, which in an event-sourced system is *safe by construction*: originals are immutable, revisions are journaled, projections rebuild. Rails apply (bounded per pass, R1-style).
 7. **Benchmark posture.** Track **LongMemEval + BEAM/ConvoMem** as the internal sanity gates; treat **LoCoMo as contested** (Zep’s critique: answer leakage, short horizons) and never headline it. Per the honesty charter, public claims remain measured-latency/SLO numbers only — these suites gate regressions, they do not market.
-

10. Phased implementation roadmap

Six waves, each with an explicit exit gate. No wave lands without (a) a benchmark delta and (b) a parity/SLO proof. Ordering favors the cheapest realized wins first — and Wave 0 precedes everything because it re-baselines the world.

Wave 0 — Execute the committed runtime flip and re-baseline — *largest single win; ops-only (new in v1.1)*

- Operator-confirmed `apply-perf-runtime.sh` run: colima 6 vCPU/12 GiB, service ceilings, host-Metal LLM default (§7.9.1). Vault unseal runbook on hand.
- Post-flip quiesced-host re-baseline of every P0 harness; consolidation per-role wall-clock captured (§7.9.2–7.9.3).
- Postgres server-tuning file sized to the resized VM (§7.9.4); before/after rows recorded.
- **Exit gate:** stack 20/20 healthy; before/after role-call and e2e numbers committed to `infra/PRODUCTION-EVIDENCE.md`; new baselines committed; rollback script verified present.

Wave A — Realize existing acceleration (native-default + caching) — *fastest engine ROI*

- Native wheels as a CI/release gate; native as the resolved default tier (§7.1.1–7.1.2).
- GIL release + bulk/packed boundary APIs; collapse the per-hit embedding pass (§7.1.3–7.1.5).
- Content-hash embedding cache + calibration cache (§7.8.1, 7.8.3), with the byte-identity cache-safety test and honeypot seeding.
- Parallel channels default-on for `standard / accelerated` (§7.5.1).
- **Exit gate:** `test_native_parity.py` green; committed baseline gates hold; native-vs-pure and cache-hit-vs-miss benchmarks recorded; warm P95 re-measured on `eval/latency`; zero SLO regression.

Wave B — Provider layer + Postgres candidate generation — *biggest latency/throughput win*

- Provider bake-off: `services/embedding` vs TEI vs `mname-providers` (defects fixed) vs in-process ONNX; contract conformance suite in CI (§7.2.1–7.2.4, §7.11.1).
- Null-embedding backfill + fallback cap; `'none'` -partition index coverage; prepared statements; per-route `ef_search`; iterative scans; `halfvec` index + exact rescoring (§7.4).
- Result cache with full-context key + write invalidation (§7.8.2).
- **Exit gate:** live Postgres lanes green; recall@k/nDCG hold under APPROX changes; “same model” vectors proven byte-identical; vector-index total-coverage drift check green; throughput + P50/P95/P99 deltas recorded; ECE re-proved ≤ 0.05 for any model/precision change.

Wave C — Structural refactor + consolidation + journal package — *durability and maintainability*

- `cli.py` decomposition seam-by-seam behind the argparse facade; deepen lazy imports; warm MCP reuse (§7.7).
- Consolidation role parallelism + batching + bounded worker concurrency + sleep-time scheduling (§7.6).
- Journal & projection package: snapshots, blue/green rebuilds, micro-batched appends, gap-safe consumers; Rust fold kernel behind the bench seam (§7.10).
- Schema migration discipline; pipeline-Protocol CI guardrail (§7.11.2–7.11.3).
- Local runtime-state contention fix; `sqlite-vec` default; optional local ANN (§7.3).
- **Exit gate:** full pytest + drift checks green; CLI cold-start import time measured before/after; consolidation pass throughput measured against Wave-0 wall-clock baseline; rebuild-equivalence tests green; §31 rails + mutation budget still enforced.

Wave D — Scale & frontier options — *large-tenant and forward-looking*

- `pgvectorscale` StreamingDiskANN for large tenants; per-tenant partitioning (with shred-synergy); index-only scans (§7.4.8–7.4.9, §7.10.5).
- OPT-IN embedding-identity fix and/or Matryoshka cascade (fully recalibrated) (§7.2.6–7.2.7); tier-0 static-embedding prefilter (§7.2.8).
- Rust front-end Phase-4 gate decided with data and recorded as an ADR either way (§7.7.4).
- Free-threaded CPython evaluation lane (§8.4).
- **Exit gate:** 100k-item benchmark cells captured on the reference machine (closing the B20 gap); DiskANN recall@99% verified; any OPT-IN tier ships off-by-default with recalibration evidence retained.

Wave E — Memory-capability upgrades — *quality × speed, all additive (new in v1.1)*

- Core-profile projection + cognify buffers + verdict-event vocabulary (§9.2.1–9.2.3).
- Recency fusion leg and passage-node graph linking behind the eval gate (§9.2.4–9.2.5); memory-evolution events (§9.2.6).
- KV-cache activation prewarm measured on the flipped runtime (§7.8.6).
- LongMemEval/BEAM internal gate wired into `eval/` (§9.2.7).
- **Exit gate:** golden-scenario suite green; every new leg/projection default-off with its ablation number recorded (the `eval/g0` gate discipline: pre-register the metric, ship only if the target improves and no guardrail regresses); honeytokens seeded in every new projection/cache; shadow-cognitive scope decision recorded (§7.11.5).

11. Benchmark & verification plan

- Performance claims are worthless without a repeatable harness. This plan extends the existing eval infrastructure rather than inventing a new one.
1. **Warm long-lived-server bench (primary).** `eval/latency/bench.py --clients N --queries M` is already the correct instrument (it fixed the Wave-2 subprocess artifact). Standardize a matrix: {local, sqlite, postgres} × {pure, native} × {no-cache, cache} × {serial, parallel-channels} × {current-embedder, TEI, mneme-providers, in-process-ONNX}. Record P50/P95/P99 + bootstrap 95% CI + throughput for `embed_call_only`, `engine_only`, `fast_path_total`.
 2. **Parity gate (blocking).** `tests/test_native_parity.py` and the shared engine contract must be byte-identical after every PARITY-SAFE change. For “same model” provider swaps, assert vector-level equality. This gate blocks merge.
 3. **Quality gate (blocking for APPROX changes).** `eval/` `recall@k` / `nDCG@k` must hold within tolerance for every APPROX-LAYER change (`ef_search`, iterative scans, halfvec, DiskANN, local ANN, prefilter legs, passage nodes). ECE must be re-proved ≤ 0.05 for every OPT-IN/RECAL change.
 4. **Cold-start bench.** Measure `python -m mnemosyne.cli ... --help` and a representative one-shot command import time before/after the CLI decomposition and lazy-import deepening; keep the MCP trace triple (spawn/init/first-call) current.
 5. **Concurrency/throughput bench.** Sustained QPS with pooled Postgres + dynamic-batched embedder; watch P99 tail and the null-embedding counter (must be 0 on the hot path).
 6. **Regression guard.** The committed baseline gates (`tests/benchmarks/test_retrieval_baselines.py` with 10×/3× exit gates) stay blocking; add a `pytest-benchmark` ratchet so a future change that regresses a proven speedup fails CI. Treat committed `*_latest.*` reports as historical (per CONCERNS) — always rerun on the target hardware and record hardware context.
 7. **Subagent verification.** Before ratifying any wave’s exit gate, run an independent review pass (fresh agent) over the diff + benchmark artifacts to confirm the parity tag on each change is correct and no oracle numeric path was touched.
 8. **Measurement-gap register (must close; new in v1.1).** The following are currently *asserted but unmeasured* — each blocks the decision that depends on it: (a) **100k-item retrieve cells** per engine (blocks the native-accel P6 ANN/quantized-tier decision — the spec defers it to a reference-Mac nightly that has no committed report); (b) **post-flip runtime numbers** (the 48–55× role-call and consolidation wins are projected until Wave 0 measures them); (c) **mneme-providers bake-off** (zero recorded numbers; blocks §7.2.2); (d) **consolidation per-role wall-clock** (P0 line item, no committed result file); (e) **Rust front-end spawn-latency delta** (blocks the Phase-4 gate); (f) **embed round-trip single vs batch** and **per-container CPU-seconds during consolidation** (P0 line items, unlocated). Additionally, adopt a **quiesced-host protocol** for every e2e run — the 2026-07-05 delta report documents ambient-load contamination (~10 LA, p95 tails inflated up to +252% on untouched paths); record load average with every artifact and reject contaminated runs.

12. Risk register & parity guardrails

Risk	Likelihood	Impact	Mitigation
An “optimization” silently changes oracle numerics	Med	High (breaks parity proof + ECE)	Every change carries a parity tag; blocking byte-identity gate; subagent review of diffs
Caching leaks across a sensitivity/branch/as-of boundary	Med	High (privacy/§31 R6)	Full-context cache keys; cache post-sanitization/post-redaction only; dedicated boundary test; honeytokens in every cache namespace
Wave-0 flip destabilizes the prod stack (Vault reseal, container churn)	Med	Med (recoverable)	Guarded script + rollback pair; unseal runbook; flip only with operator present; evidence captured before/after
Provider colocation reintroduces an implicit network/model dependency	Med	Med (breaks local-first)	Opt-in only; deterministic local fallback stays canonical default
CLI decomposition regresses release/evidence gates	Med	High (false-completion controls)	Seam-by-seam, each pinned by a focused test; change one evidence invariant at a time

Risk	Likelihood	Impact	Mitigation
halfvec/DiskANN degrade recall under filters	Low–Med	Med	recall@k/nDCG gate on eval; iterative scans; exact rescoring at apex
AGPL/ELv2 code (VectorChord, pg_search) contaminates the distribution posture	Low	Med (legal/audit)	§9.1 license gate; adopt algorithms, not extensions; ADR required for any exception
New derived structures (ANN cache, profile projection, sketches) drift from the ledger or survive a shred	Low–Med	High (correctness/GDPR)	Derived-structure rule (§9.1): rebuildable, fingerprint-stamped, shred-rebuild tested, honeytokens-seeded
Consolidation role parallelism perturbs role ordering / rail budgets	Med	Med	Fixture-pin current behavior first; parallelize only proven-independent roles; rails asserted in tests
Gap-unsafe projection consumers skip events under load	Med	High (silent projection drift)	§7.10.6 sequencer/gap-detection; rebuild-equivalence tests; projection fingerprints
Native wheel build breaks on a platform in the matrix	Med	Low (pure-Python fallback)	Pure-Python remains canonical; wheels gated but fallback guaranteed
Free-threaded CPython instability	Low	Low (opt-in lane)	Kept as future tier behind its own CI lane; abi3-py312 stays canonical
Parallel channels introduce nondeterministic fusion	Low	High (parity)	Canonical fixed consume-order; shuffle-completion test asserts identical output
Two performance programs drift apart (this blueprint vs the P0–P6 spec lane)	Med	Med (duplicated/conflicting work)	§4.6 is normative: this blueprint extends the standing programs; any conflict resolves in favor of the dated spec + a reconciling edit here

Standing guardrail. Every merge in this program runs: (1) parity byte-identity gate, (2) `eval` SLO gate, (3) full pytest + config-drift checks, (4) for APPROX/RECAL, the quality/ECE gate. A change that cannot state its parity tag and pass its gate does not merge.

13. Appendix A — File-level change map

File / dir	Change	Section
<code>infra/scripts/apply-perf-runtime.sh</code> (run), <code>infra/PERF-RUNTIME.md</code> , <code>infra/PRODUCTION-EVIDENCE.md</code>	Execute Wave-0 flip; record before/after evidence	§7.9
<code>infra/postgres/postgresql-perf.conf</code> (new), <code>infra/docker-compose.prod.yml</code>	Version-controlled server tuning sized to resized VM	§7.4.7, §7.9.4
<code>rust/mnemosyne-native/src/*.rs</code>	GIL release (<code>Python::detach</code>), packed/bulk entry points, zero-copy buffers, projection-fold kernel; no float-op changes	§7.1, §7.10.3
<code>.github/workflows/ci.yml</code>	Promote <code>native-wheels</code> + parity to required gates; benchmark ratchet lane; provider conformance lane; pipeline-Protocol structural check	§7.1, §7.11, §11
<code>src/mnemosyne/capability.py</code>	Resolve native as default tier; parallel channels default-on for standard/accelerated	§7.1, §7.5
<code>src/mnemosyne/retrieval.py</code>	Content-hash embedding cache; dynamic request batching; in-process ONNX provider; batched rerank; tier-0 prefilter leg (opt-in)	§7.2, §7.8
<code>src/mnemosyne/pipeline.py</code>	Parallel channels default; overlap embed with lexical/graph; deterministic fusion test hook; optional recency leg (opt-in)	§7.5, §9.2.4
<code>src/mnemosyne/postgres_engine.py</code>	Null-embedding backfill + cap; prepared statements; per-route <code>ef_search</code> ; iterative scans; halfvec index + exact rescore; partition-coverage fix; stored-vector MMR default path (opt-in); pgvectorscale adapter	§7.4, §7.5.5
<code>src/mnemosyne/sqlite_engine.py</code>	<code>sqlite-vec</code> default; read pragmas/mmap; optional local ANN	§7.3
<code>src/mnemosyne/engine.py</code>	Collapse per-hit embedding pass into packed path; local ANN candidate generator (corpus-size gated); route→ <code>ef_search</code> wiring	§7.1.4, §7.3, §7.4
<code>src/mnemosyne/journal.py</code> , <code>src/mnemosyne/projections.py</code> , <code>src/mnemosyne/jobs.py</code> , <code>src/mnemosyne/queue.py</code>	Snapshots; blue/green rebuild machinery; micro-batched appends; gap-safe checkpointed consumers	§7.10
<code>src/mnemosyne/runtime_state.py</code>	Write-behind/serialized metric persistence	§7.3
<code>src/mnemosyne/consolidation.py</code>	Role parallelism (fixture-pinned); batched embeds; cognify buffers; verdict-event vocabulary; sleep-time scheduling	§7.6, §9.2.2–9.2.3

File / dir	Change	Section
src/mnemosyne/cli.py → cli_ops/ , cli_evidence/ , cli_runtime/	Decompose behind argparse facade, seam-by-seam	§7.7
src/mnemosyne/mcp_server.py	Warm engine reuse / keepalive; avoid per-call rebuild	§7.7
services/embedding/ , rust/mneme-providers/	Provider bake-off (fix sidecar mutex + single-input defects first); TEI deploy path; unified contract + conformance suite	§7.2, §7.11.1
sql/migrations/ (new), sql/schema.sql	Migration discipline; halfvec columns; 'none' -partition coverage; covering indexes; per-tenant partitioning; DiskANN DDL (opt-in)	§7.4, §7.10.5, §7.11.2
eval/latency/ , eval/ , tests/	Benchmark matrix; parity/quality/cache-safety gates; cold-start + concurrency benches; measurement-gap closures; LongMemEval/BEAM internal gate	§11, §9.2.7
docs/adr/ , docs/decisions/	ADRs for: Rust front-end verdict, index/partition strategy, embedding-model identity, shadow-tier scope, any AGPL exception	§4.6, §7.7.4, §7.11.4–7.11.5, §9.1

Appendix B — Environment knobs touched (register in `CONFIG-DRIFT-CHECKS.md`)

`MNEMOSYNE_PURE` (parity escape hatch, unchanged), `MNEMOSYNE_PARALLEL_CHANNELS` (default-on policy per tier), `MNEMOSYNE_EMBED_BATCH_SIZE` (tier-tuned, unchanged), `MNEMOSYNE_CAPABILITY_TIER` / `MNEMOSYNE_CAPABILITY_AUTOTUNE` (unchanged), plus new: content-hash embedding-cache size/TTL, result-cache size/TTL, embedder provider selector (current/TEI/mneme-providers/onnx), `hnsw.ef_search` route mapping, halfvec/iterative-scan toggles, pgvectorscale enable, recency-leg toggle, tier-0 prefilter toggle, profile-projection enable, KV-prewarm enable, snapshot cadence, micro-batch window. Every new knob must default to current behavior and be documented, per the config-drift discipline.

Appendix C — Open questions for ratification

Routed through the SECTION-17 decision-memo / ADR pattern; none reopen settled decisions.

1. Which platforms enter the required native-wheel matrix first (Linux x86_64 + macOS arm64 is the minimum viable pair)?
2. Provider primary: TEI, the repaired `mneme-providers`, or in-process ONNX for single-node self-hosted — decided by the Wave-B bake-off, recorded as an ADR.
3. Should the result cache be in-process only, or backed by Postgres for multi-node MCP fleets (with the same full-context key + invalidation contract)?
4. Target corpus size threshold at which the local ANN and pgvectorscale paths switch on?
5. Embedding-model identity (§7.2.6): keep bge-small+projection, store native 384d, or adopt an MRL model — decided by bake-off + recalibration evidence.
6. Index/partition strategy for total vector coverage (§7.11.4): third partial index, guaranteed-partition ingest, or single covering index + iterative scans?
7. Shadow-cognitive tier scope (§7.11.5): keep / freeze / extract?
8. Do we commit to a free-threaded CPython lane this cycle, or defer to the next?
9. Wave-E capability upgrades: which of the profile projection, recency leg, and passage-node linking are in scope for this cycle vs the next?

This blueprint is a proposal. It makes no production claim. Every projected figure must be proven by the §11 benchmark plan and pass the §12 guardrails before any status or parity document is updated, per `.planning/ACTIVE-GOAL-OPERATING-CONTRACT.md`.